



Lecture 01b: The Abstract Syntax Tree (AST) Deep Dive

INTERVIEW QUESTION

What is an Abstract Syntax Tree (AST), and how does Svelte's use of ASTs differ from how React and Vue use them?

(Note: This is an advanced conceptual question. Knowing the answer elevates you from a standard "framework user" to a developer who understands computer science fundamentals.)

When we talk about a compiler "parsing" your code, we are actually talking about the creation of an **Abstract Syntax Tree (AST)**. An AST is a universal concept in computer science. Almost every programming language, compiler, or interpreter relies on it.

When you write code, you see it as a flat string of text (e.g., `let x = 5;`). But a computer cannot reason about a string of text. To process your code, a parser breaks that text down into a giant, branching tree structure of objects.

KEY TAKEAWAY

What the DOM is to HTML, the AST is to code.

Just as the browser parses a flat HTML string into a Document Object Model (DOM) tree so it can manipulate the web page, a compiler parses a flat string of code into an Abstract Syntax Tree (AST) so it can analyze and manipulate the program.

For example, `1 + 2` becomes a tree where:

- The top "Node" is a `BinaryExpression` (Addition)
- The left "Branch" is a `NumericLiteral` (1)
- The right "Branch" is a `NumericLiteral` (2)

This tree structure allows a program to systematically walk through your code, understand its logic, and translate it into something else.

Do React and Vue use ASTs?

Yes, they absolutely do! But they use them very differently than Svelte.

1. React

When you write React, you write JSX (like `<h1>Hello</h1>`), which is not valid JavaScript. Before your code hits the browser, a build tool (like Babel or SWC) uses an AST to translate that JSX into standard JavaScript functions (like `React.createElement('h1', null, 'Hello')`).

However, that is where React's compile-time work stops. At *runtime* in the browser, React must rely on the **Virtual DOM** to figure out what data changed and what physical pixels to update.

2. Vue

Vue works similarly. A Vue compiler reads your `.vue` templates, builds an AST of your HTML, and translates it into JavaScript render functions. But just like React, Vue still relies

heavily on a Virtual DOM at runtime to apply those updates.

The Svelte Difference

React and Vue use ASTs primarily to *transpile their syntax* into a format the browser can run.

Svelte goes much deeper. Svelte uses the AST to fundamentally **map out the logic of your application**.

Because Svelte traces exactly where a variable is declared and exactly what HTML element it touches inside the AST, it can skip the Virtual DOM entirely. It uses the AST to write the exact, direct DOM update instructions (e.g., `h1.textContent = newValue`) straight into the final standalone JavaScript bundle.

INTERVIEW TIP

To impress the interviewer: During an interview, most candidates will simply state that "Svelte doesn't use a Virtual DOM." If you want to show deep technical background and skills, explain *why* it doesn't need one: *"Svelte doesn't need a Virtual DOM at runtime because it uses an Abstract Syntax Tree at compile-time to permanently map the relationship between state and the DOM."* This proves you understand the underlying mechanics of compilers and aren't just reciting marketing material.

Summary

An AST is a tree representation of your code's structure. While React and Vue use ASTs simply to translate custom syntax into Virtual DOM instructions, Svelte uses the AST to permanently trace reactivity and generate direct DOM updates, completely eliminating the need for a runtime diffing engine.

React	Transpiles JSX into JS functions	Virtual DOM diffing engine
Vue	Transpiles templates into JS render functions	Virtual DOM diffing engine
Svelte	Maps state directly to HTML elements	Direct DOM updates (No Virtual DOM)